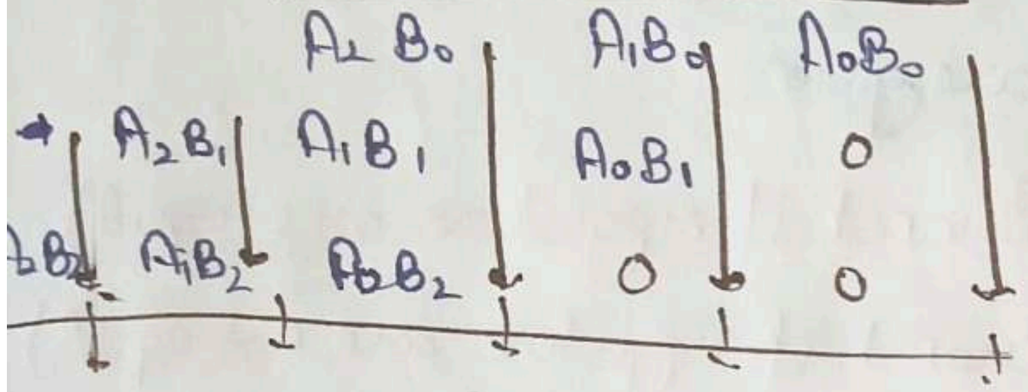
Problem-2 | Add and Shift Multiplier

In normal binary multiplier, we simply multiply each bit of the multiplier (Q) with the multiplicand (M) and shift each partial product one bit left as that of other and then add them to get the final multiplication.

A_2 A_1 A_0 (Multiplicand) ⇒ "(M)"

× B_2 B_1 B_0 (Multiplier) ⇒ "Q"



This addition will give the result.

What happens in hardware ?

In hardware implementation, we need an optimized form of multiplication for getting maximum speed and minimal circuit area.

In this Shift and Add method (the optimized multiplier schematic) for hardware implementation, it is better to shift the partial sum right by one bit and better to keep the partial product at fixed position.

Working of optimized Multiplier

Suppose we want to multiply two 3-bit numbers '101' and '110'

'101' is Multiplicand,
'110' is Multiplier.,

```
    101
  x 110
  -----
  000000   (Initially the partial sum will be zero)

+ 000 (Product of LSB)   { we have taken 6 bits of partial sum
  -------                   bcz the ans will contain 6 bits }
  000000   (1st partial sum)
```

↳ Shift right by one bit !!

```
  000000
+ 101   (Product of 2nd bit)
  ------
  101000   (2nd partial sum)
```

↳ Shift Right

```
  010100
+ 101   (Product of 3rd bit)
  ------
  111100   (3rd partial sum)
```

↳ Shift right.

011110 Ans

∴ (101) × (110) = (011110)

↳ Find answer is computed after shifting the last partial sum.

Implementation in Hardware

For multiplying two, 4-bit numbers, we will be needed three 4-bit registers.

A (Accumulator)

Q (Multiplier)

M (Multiplicand)

→ The Accumulator (A) is used for storing the partial sum.

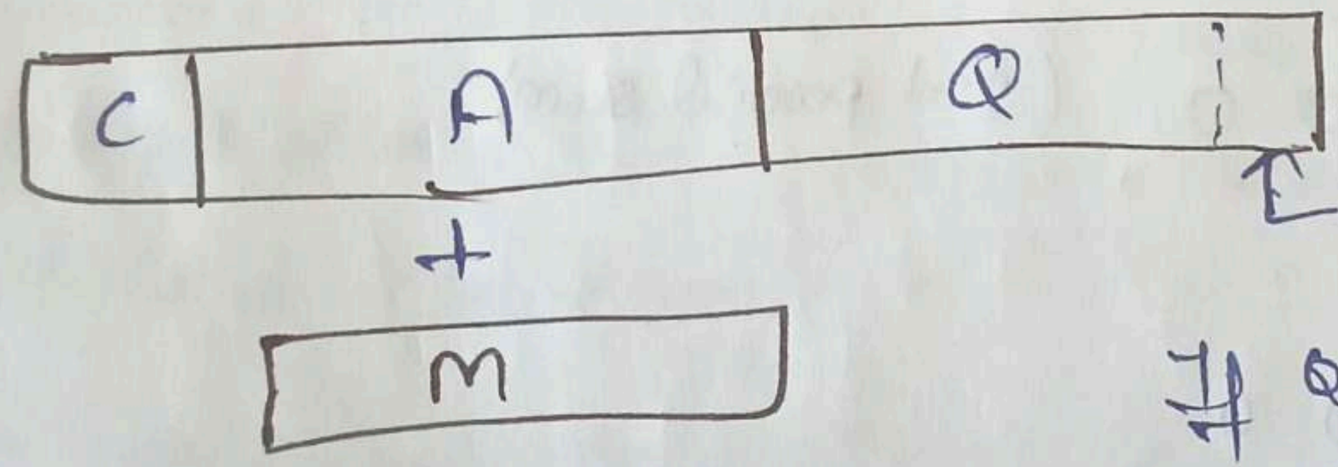
Rule for the multiplication,

If bit of the multiplier is → 0, product = 0

→ 1, product = M

∴ Everytime after shifting we will compare the LSB of the Multiplier.

C → Carry bit



If $Q_0 = 0$, $M = 0$

$Q_0 = 1$, $M = 1$

C, A and Q will be right shifted and M will be added based on Q_0.

Example ①| 1011 × 0111 (11×7 = 77)

Q = 1011, M = 0111

Initially A = 0000, C = 0

C	A	Q		
0	0000	1011 ↑LSB		
+	0111		(M=M)	
0	0111	1011	A = A+M (Add)	1st cycle
0	0011	1101 ↑LSB	(Right shift)	
+	0111		(M=M)	
0	1010	1101	(Add)	2nd cycle
0	0101	0110 ↑LSB	(shift)	
+	0000		(M=0)	
0	0101	0110	(Add)	3rd cycle
0	0010	1011 ↑LSB	(shift)	
+	0111		(M=M)	
0	1001	1011	(Add)	4th cycle
0	0100	1101	(shift)	

"Product"

∴ Ans = $(0100\ 1101)_2$

or

$(77)_{10}$

Example ②) 0110 × 1001 (6 × 9 = 54)

M = 0110, Q = 1001

Initially A = 0000, C = 0

C	A	Q		
0	0000	1001		
			↳ LSB = 1 ⇒ M = M	
+	0110			
0	0110	1001	(Add)	} 1st cycle
0	0011	0100	(Right Shift)	
+	0000		↳ LSB = 0	
0	0011	0100	(Add)	} 2nd cycle
0	0001	1010	(Shift)	
			↳ LSB = 0	
+	0000			
0	0001	1010	(Add)	} 3rd cycle
0	0000	1101	(Shift)	
			↳ LSB = 1	
+	0110			
0	0110	1101	(Add)	} 4th cycle
0	0011	0110	(Shift)	

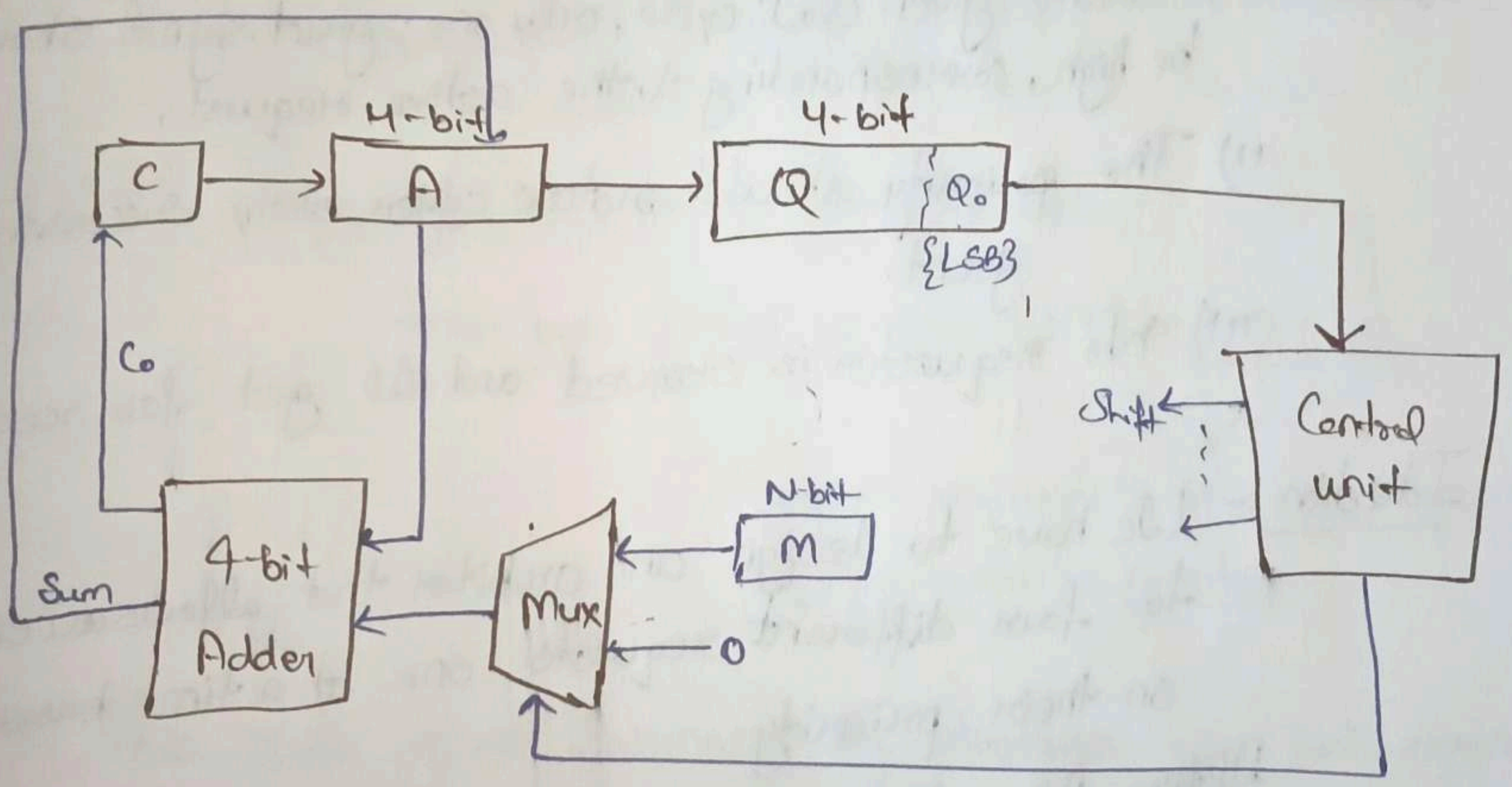
Product

Ans = $(0011\ 0110)_2$ or (54)$_0$

Schematic idea & Module declaration

- A and Q should be 4-bit shift registers
- To store the carry bit, one additional flip-flop will be required
- To perform the addition we need 4-bit adder
- To control add and Shift operations & to count the number of Iterations, we will also require control unit

[1:0]count



- After 4 interations, the product is {A, Q}
(8bits)